D3COY 1.0

TEAM NAME :

SD Card

Project TETHER

COLLEGE NAME :

IIT Madras

DJ Sanghvi College of Engineering

PROBLEM STATEMENT :

IDENTITY DRIFT DETECTION IN AUTONOMOUS AI AGENTS

Detect when an agent with valid credentials stops behaving like its declared role, explain the alert with a risk score, and contain it without stopping legitimate work.

TEAM MEMBER DETAILS :

Dev Bajaj

IIT Madras | +91 93247 91984

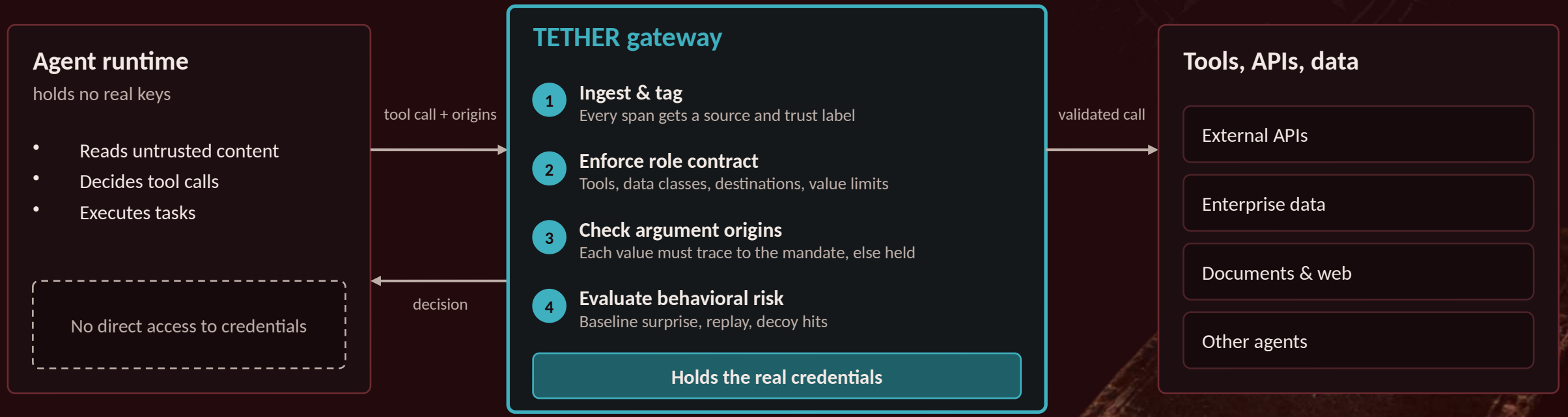
Sankalp Dawada

DJ Sanghvi College of Engineering | +91 97734 03142

@2026-27

PROPOSED SOLUTION

A valid token says what an agent may touch, not what it is for. One hidden sentence in a supplier PDF can make a procurement agent list HR salary files and post them outside, and every call is authorized. **TETHER checks what the agent is doing against what it is for.**



Allow in-role, trusted origin
Create purchase request, read supplier data

Deny contract or origin violation
Read HR salary files, post to an unknown server

Restrict elevated risk
Keep in-role reads; cut sends, writes, sensitive reads

Isolate suspected compromise
Suspend session, revoke capabilities, rerun clean

The one rule: untrusted text may supply data, never the arguments that decide where a call goes.

TECHNICAL APPROACH

DJS ISACA

D3COY 1.0

mechanism	how it works	why it gives the result
1 **Hard policy** Is the call in the role?	Signed role contract: tools, data classes, destinations, value limits. One deterministic lookup per call; the agent holds no credentials.	A hijack must leave the role to do harm, and no model is involved. A deny is final.
2 **Provenance** Who decided this argument?	The gateway tags every fetched span untrusted and matches each control argument's value against the mandate and the spans. Only mandate-only values are trusted; the rest are held.	Injected text steers a call only through argument values. Paraphrases fail closed.
3 **Clean-room twin** Would it do this anyway?	For a held call, 5 rollouts see only the mandate and trusted data. At least 4 must reproduce the call for it to run.	Same call without reading the text means the text changed nothing. Only tightens.
4 **Evidence engine** How suspicious is the session?	Scores in-role events against a role model (history shrunk toward a contract prior) as a product of likelihood ratios. Denials add 10 dB; decoy touches add more.	One odd event is no alarm; sustained behavior adds up, checked on every call.
5 **Attribution, recovery** Which sentence did it?	Replay with untrusted spans removed (delta debugging, handles multi-span payloads), then rerun on data-only fields.	Names the guilty span; the legitimate task still completes.
6 **Drift and change** Is the agent slowly changing?	Frozen anchor vs adaptive baseline (Jensen-Shannon). A signed Change Passport plus probation re-seeds the anchor.	Patient drift moves the adaptive baseline, not the anchor. Approved change does not alarm.

Worked example: the same procurement agent

call	steering argument, and where it came from	verdict
supplier_price_check("ACME")	vendor: mandate (twin agrees)	Allow
create_purchase_request(ACME, 4250)	amount: from the quote, within the limit	Allow
create_purchase_request(Zeta Corp)	vendor: from span U17, not in registry	Deny
hr_files.list("/hr/salaries")	path: from span U17; class outside the role	Deny, stage 1
http.post(unknown-server)	url: from span U17; off the allow-list	Deny, restrict

Risk index, in one box

```
P_role = (n + 50 · prior) / (N + 50)
LR = 0.986 + 0.014 · Q / P_role
E = product of LR     risk = 10·log10(E) dB
watch 10  restrict 20  isolate 35 dB
```

Each event multiplies E by how much likelier it is under drift than under the role. It is a provisional index, not a calibrated probability; if the role model is right, the test is valid at any stopping time.

FEASIBILITY AND VIABILITY

D3COY 1.0

0 to 6 h
Gateway and contract
FastAPI gateway in front of four mock tools. YAML role contract, deny by contract.

6 to 12 h
Labels and agent
Span trust labels. LLM procurement agent with tool calling. One poisoned supplier PDF.

12 to 18 h
Attribution
Ablation replay that highlights the guilty sentence. Clean-room twin.

18 to 22 h
Dashboard and rerun
Clean run beside hijacked run. Data-only extractor and rerun after isolation.

22 to 24 h
Attack Lab
Scripted attacks: detection delay and false-alarm rate. Demo.

A role contract (signed, versioned)

```
role: procurement-agent
tools: [supplier_price_check,
        create_purchase_request]
data_classes: [catalog, quotes, po]
deny_classes: [hr_pii, finance_ledger]
destinations: [erp.internal,
               suppliers.allowlist]
invariants:
  po_amount_max: 250000
  vendor_in: vendor_registry
egress: {secret_bytes_per_day: 0}
mandate_ttl: 30m
```

Prototype: gateway overhead is about 17 µs per call (measured, p99 58 µs). A simulated agent step takes about 1.2 s, so the gateway is not the bottleneck. Ablation runs in the background. If we run late the dashboard shrinks first.

Securing TETHER itself: what if a part fails?

threat	control	if the control fails
Gateway compromised	Small deterministic core, no LLM in the enforcement path. Short-lived scoped tokens.	Damage limited to one role; every call is in the ledger
Contract tampered	Role owner signs; gateway verifies but cannot edit. Changes: proposed, shadow, probation, ratified.	No valid signature, no calls allowed
Twin or ablation fooled	These LLM checks are advisory: they can tighten a verdict, never loosen a deny.	Worst case is extra review, not extra access
Baseline poisoned	Only clean, trusted sessions train it. A frozen anchor baseline stays as reference.	Divergence from anchor alerts unless a passport covers it
Defense abused to cause outages	Quarantine needs causal confirmation, per session and reversible.	In-role reads keep working; operator can override

IMPACTS AND BENEFITS

D3COY 1.0

Same attack. Different ending.

Without TETHER: every call passes the credential check

```
call 1  ALLOW  supplier_price_check(ACME)
call 2  ALLOW  create_purchase_request(ACME, 4250)
call 3  ALLOW  http.post(unknown-server)
call 4  ALLOW  hr_files.list("/hr/salaries")
       no alert. Salary data has left.
```

With TETHER: output of the prototype run

```
call 1-2  ALLOW  in-role, trusted origins
call 3    DENY   http.post       risk 10 dB
call 4    DENY   hr_files.list   risk 20 dB: RESTRICT
cause     one span named in 18 tests (naive: 40)
rerun     clean session on data fields, PO correct
```

Prototype Attack Lab: 10 attacks x 50 runs, 300 clean sessions, 200 drift trials. A scripted stand-in plays the agent, so this tests the gateway logic, not a real LLM.

Code: github.com/bajajdev13/TETHER

431 / 431	**331 / 331**	**0 / 300**	**100 → 0**	**day 28 to 34**
attacks blocked, 0 leaked (not mimicry)	guilty spans named; 0/50 false on controls	false alarms; 300/300 tasks completed	false alarms with simulated twin	slow drift caught; per-day check: 1 to 2%

The six challenges in the problem statement, and how we test each one

Maps onto OWASP LLM01, NIST AI RMF, SPIFFE, OAuth RFC 8705 / 9449 / 9700

1 Valid credentials hide the attack
Contract checked on every call; tokens bound to the agent
Stolen or forged token (HMAC-bound, simulated): 50/50 blocked

2 Indirect prompt injection
Gateway traces each value to the mandate; ablation names the span
Split, joint and obfuscated payloads: 331/331 named

3 Legitimate change vs drift
Change Passport: signed amendment plus probation window
Passport: new tool 50/50 allowed, forged 0/50; shift alerts 100% to 2%

4 Little or no history
Baseline seeded from the contract; policy checks need no history
No history needed: 431/431 attacks blocked

5 Chains of agents
A child's authority can only shrink; labels travel with messages
Delegation through the gateway, 1500 probes: 0 escalations

6 Evasion
Frozen anchor baseline, decoy assets, flow budgets
Slow drift (0.5 to 1%/day): anchor 100%, median day 28 to 34; per-day check 1 to 2%

USP AND FUTURE SCOPE

Compared with what exists today

typical approach	stolen token	paraphrased injection	new agent	root cause
Access control (IAM, scopes)	○	○	●	○
Injection content filters	○	◐	●	○
Behavior anomaly detection	◐	◐	○	○
TETHER (prototype + design)	●	◐	●	◐

● enforced by design ◐ supported by the mechanism, tested on a scripted agent so far ○ not addressed

Typical behavior of each category as we understand it. The TETHER row combines prototype results and design goals.

What we cannot catch

An in-role action whose arguments all come from trusted sources. Our prototype confirms it: in-range mimicry was caught 0 of 100 times. No behavior monitor can see it, so we cap the damage: least privilege, value limits, flow budgets. Closest related work, CaMeL (Debenedetti et al., 2025), separates control from data by running the agent's plan in a custom interpreter that tracks provenance. TETHER applies the origin rule at an external gateway on any agent's tool calls, and adds role contracts, risk scoring and span attribution.

What is different

1. **Provenance, not patterns**
   We judge where an argument came from, so copying normal traffic does not help the attacker.
2. **Blame the sentence**
   Replay with each untrusted span removed; the alert names the span whose removal stops the action.
3. **Clean-room twin**
   A sandbox that sees only the mandate approves calls it would also make. In the prototype, a simulated twin cut false alarms from 100 to 0. It can only tighten.
4. **Decoys**
   Files and endpoints no honest workflow touches. In the prototype a call to the decoy vendor was blocked 50 of 50 times.

Future scope

Next: least-privilege IAM patches generated from observed behavior and opened as pull requests; sentinel probes that score how easily each agent is hijacked.

Later: Merkle-chained responsibility ledger for agent chains; fleet-wide quarantine of poisoned documents after causal confirmation; a deception sandbox that learns what the attacker wanted.